



Lecture 49: Syncing Local Variables with Incoming Props

INTERVIEW QUESTION

How do you initialize a reactive local variable that dynamically syncs with an incoming prop using Runes?

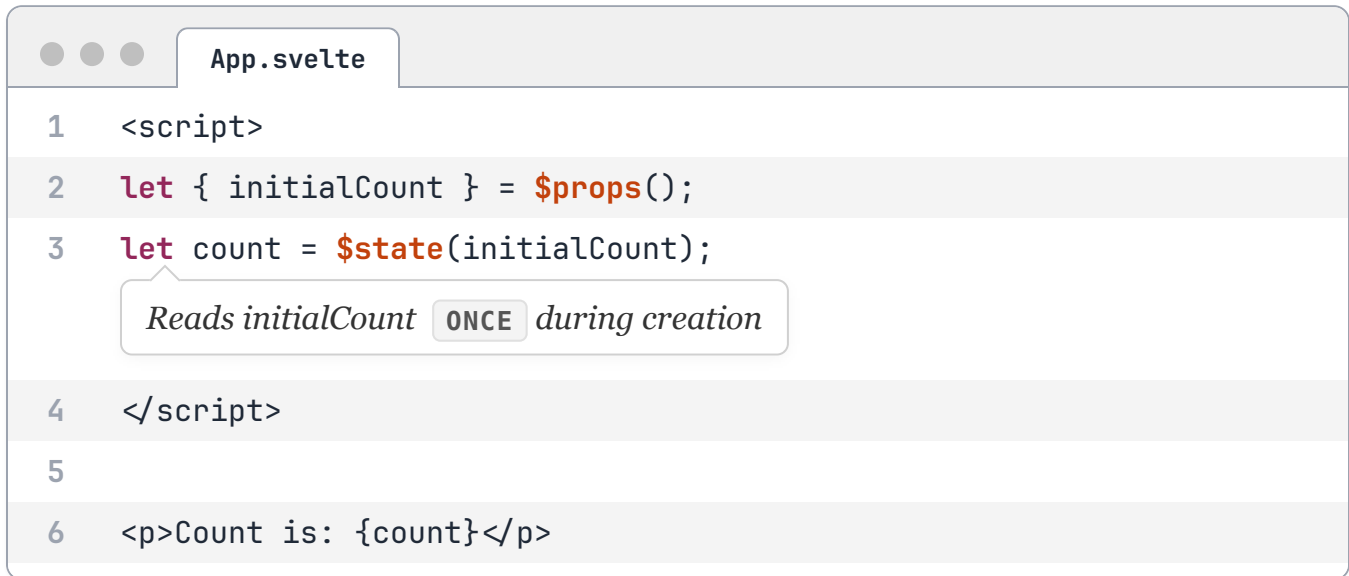
A common source of confusion when moving from legacy Svelte to Svelte 5 runes is how to handle a local variable that needs to initialize from an incoming prop, but also needs to stay in sync when that prop changes over time.

If you pass a prop down to a child component and assign it to a `$state` rune — the rune used to declare reactive state — it will capture the *initial* value of that prop perfectly. However, if the parent later updates the prop, the child's `$state` variable will stubbornly hold onto the original value. The connection is severed the moment the child component is created.

To solve this, we rely on the `$derived` rune — a reactive declaration that automatically recalculates its value whenever its dependencies change.

The problem with `$state`

When a component is instantiated, code inside the `<script>` block runs exactly once.



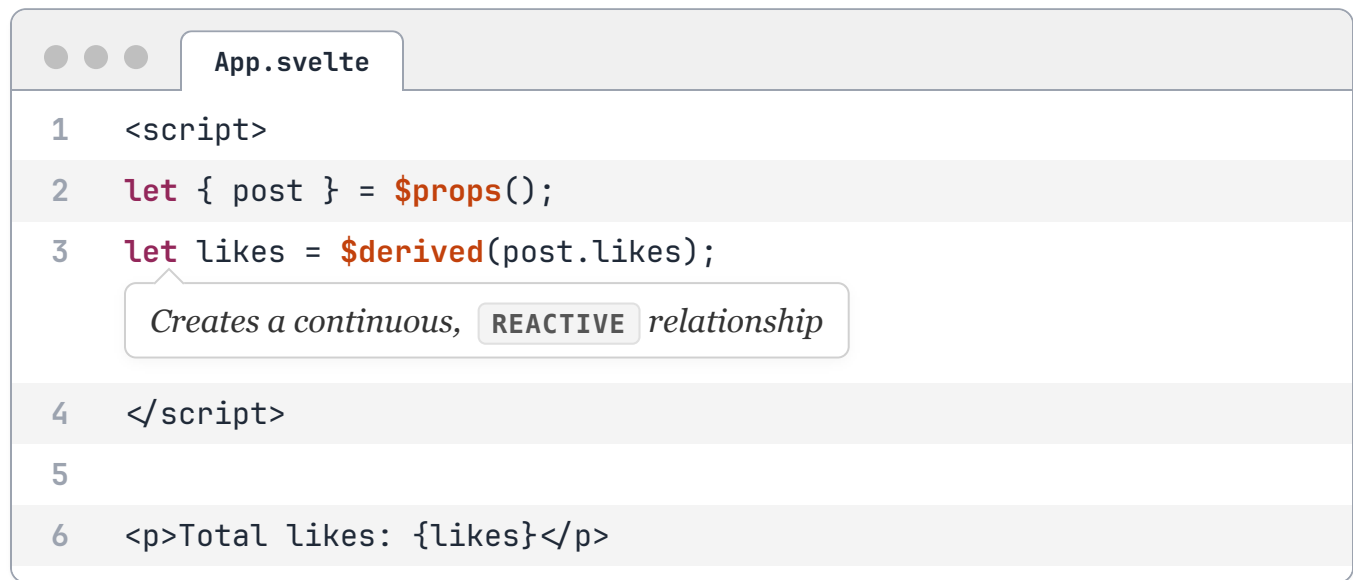
```
1 <script>
2 let { initialCount } = $props();
3 let count = $state(initialCount);
4 </script>
5
6 <p>Count is: {count}</p>
```

Reads initialCount ONCE during creation

In the example above, `count` is a completely independent variable. It took `initialCount` as its starting value, but it has no ongoing relationship with it. If the parent component changes `initialCount`, the child component will ignore the change, and `count` will not update.

The solution: `$derived`

If you want a variable to dynamically sync with a prop, you declare it as derived state. Anything read synchronously inside a `$derived` expression is considered a **dependency** — a reactive value that Svelte watches for changes. When the dependency changes, the derived state is marked as **dirty** (meaning it's stale and needs recalculation) and will be updated automatically.



```
1 <script>
2 let { post } = $props();
3 let likes = $derived(post.likes);
4 </script>
5
6 <p>Total likes: {likes}</p>
```

*Creates a continuous, **REACTIVE** relationship*

Because `$derived` expressions are re-evaluated when their inputs change, `likes` will always stay in sync with `post.likes`. Whenever the parent updates the `post` prop, `likes` recalculates.

Overriding derived values for Optimistic UI

Historically, derived values were strictly read-only. You could not assign a new value to `likes = 5`. But starting in Svelte 5.25, you can temporarily override the value of a `$derived` variable by simply reassigning it (as long as you didn't declare it with `const`).

This is specifically designed for **optimistic UI** — a pattern where you update the user interface immediately before the server confirms the change, making the app feel lightning fast.

Here is how it works in practice:

● ● ● App.svelte

```
1 <script>
2 let { post, like } = $props();
3 let likes = $derived(post.likes);
4
5 async function onclick() {
6   likes += 1;
7
8   try {
9     await like();
10
11   } catch {
12     likes -= 1;
13   }
14 }
15
16 <button {onclick}>❤️ {likes}</button>
```

Derived from the **SOURCE** of truth (the prop)

1. optimistic override: increment **LOCALLY** immediately

2. tell the server to **SAVE** the change, re-deriving likes

3. rollback: if network request failed, **REVERT** the override

When you write `likes += 1`, you are temporarily overriding the derived value. When the parent eventually passes down a new `post` object with updated likes, the `$derived` rune recalculates, and the override is washed away by the new source of truth.

Summary

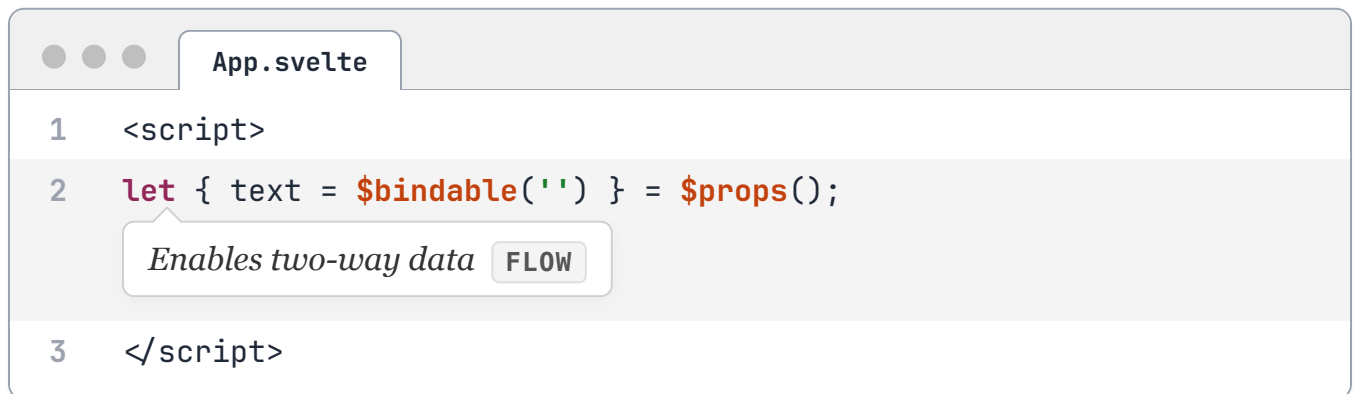
If you need a local variable that dynamically syncs with an incoming prop, do not use `$state` . Use `$derived` . And if you need to let the user temporarily update that value before the parent data catches up, you can safely reassign the derived variable.

Beyond the basics

Frequency Rating System ★★★ = Very common
gotcha / pattern ★★☆☆ = Common gotcha / pattern
★☆☆ = Niche / Advanced pattern

★★★ Two-way prop sync uses `$bindable`

The `$derived` pattern above is one-way (parent → child). When the child must also push changes back up to the parent — a text input bound to a parent-owned string, a checkbox toggling parent state — declare the prop with `$bindable()` and use `bind:text` in the parent. The Svelte docs are explicit: "don't mutate props. Either use callback props to communicate changes, or — if parent and child should share the same object — use the `$bindable` rune."



```
1 <script>
2 let { text = $bindable('') } = $props();
3 </script>
```

Enables two-way data FLOW

★★☆ Multi-statement derived values use `$derived.by`

The `$derived(expr)` form takes a single expression. If your transform needs a loop, an intermediate `let`, or a `try/catch`, switch to `$derived.by()`. Under the hood `$derived(expr)` is exactly equivalent to `$derived.by(() => expr)`; `.by` exists only for multi-statement bodies.

● ● ● App.svelte

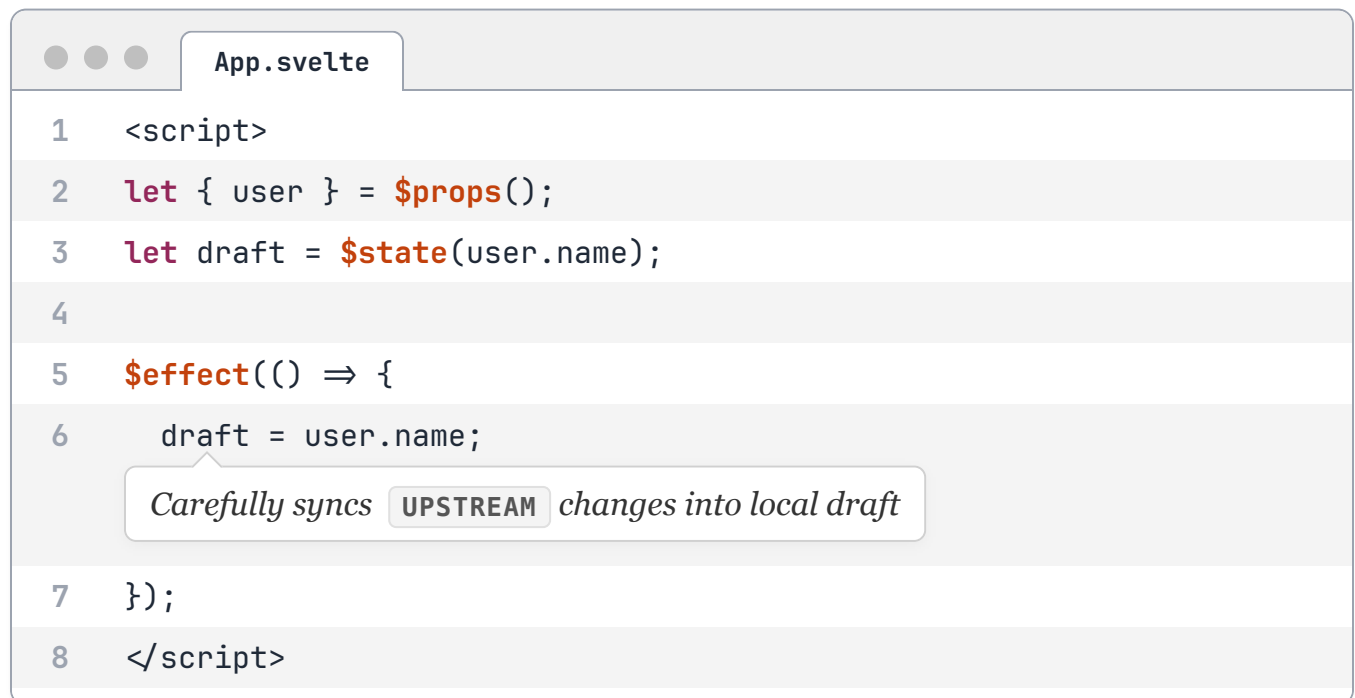
```
1 <script>
2 let formatted = $derived.by(() => {
3   let str = text.trim();
4   return str.toUpperCase();
5 });
6 </script>
```

Allows complex LOGIC inside the derivation

☆☆☆ Editable-and-syncing local drafts use `$effect`

When the child must own its own edits (a form draft the user is typing into) but also needs to re-sync when the parent hands down a new prop, the pattern is `$state(prop.value)` combined with `$effect()`. Use this sparingly —

`$effect`-driven sync can create loops if you write back to the same signal you read (see Lecture 21). Default to `$derived` first; reach for this only when editability and continuous sync are both required.



```
1 <script>
2 let { user } = $props();
3 let draft = $state(user.name);
4
5 $effect(() => {
6   draft = user.name;
7 });
8 </script>
```

Carefully syncs **UPSTREAM** changes into local draft

★★★ Destructuring a prop is a one-shot snapshot

`let { title } = $props()` reads `title` once, at component setup. Later changes to the parent's prop do not flow into the destructured `title` variable. This is the same JavaScript-level gotcha as destructuring a `$state` proxy (see Lecture 50). If you need ongoing reactivity, read through the props object: `{title}` in the template, or `let p = $props()` and then `p.title`.



```
1 <script>
2 let { title } = $props();
3 </script>
```

*Severs the **REACTIVE** connection to the prop*

★★☆ Derived expressions must be side-effect-free

The `$derived` body may not mutate state. Svelte disallows this because derived values must be pure functions of their inputs — side effects belong in `$effect`, not `$derived`. If your "sync" needs to write to another signal as part of its work, you are writing an effect, not a derived.

● ● ● App.svelte

```
1 <script>
2 let count = $state(0);
3 let doubled = $derived.by(() => {
4   count += 1;
5   return count * 2;
6 });
7 </script>
```

Mutating state here is a **COMPILE ERROR**

☆☆☆ Passing proxied data to external libraries

A `$derived` whose source is a `$state` proxy returns a proxied value. Some libraries (structured clone, certain chart libraries) trip over Proxy objects. Wrap the handoff with `$state.snapshot(derivedValue)` to pass a static deep clone instead.



```
1 <script>
2 let data = $derived(post.metrics);
3
4 $effect(() => {
5   new Chart($state.snapshot(data));
6 });
7 </script>
```

*Passes a raw, **CLONED** object*

★★★ Don't conflate the three syncing runes

Picking the wrong one is the most common source of "why isn't my UI updating" bugs in Svelte 5 components that receive props.

● ● ● App.svelte

```
1 <script>
2 let { data } = $props();
3 let a = $state(data);
4 let b = $derived(data);
5 let { c = $bindable() } = $props();
6 </script>
```

Initializes once, then goes **DEAF**

Listens forever, but is **READ-ONLY**

Listens forever AND is **WRITABLE**